Name: Taha Ahmed Mallick Roll No.: CT-25183
Sec: D

# DSA - Assignment#1

Pg#1

Note: The language used is C++ and solution may use data structures provided by STL.

## Question#"1"

```
map<int, int> nextGreaterFreq(vector<int>& nums){
//This function returns a map<int,int> where
//key is the unique element and value is the next
//greater frequency.

    map<int, int> freq; // stores freq of each unique elemen
    for(int n : nums){
        freq[n]++;
    }

    set<int> unqFreq; //removes duplicate frequency
    for(auto it : freq){
        unqFreq.insert(it.second);
    }

    map<int, int> result;
    // Finding smallest frequency greater than current
    for(auto curr: freq){
        int element = curr.first;
        int currFreq = curr.second;

        auto it = unqFreq.upper_bound(currFreq);

        if(it == unqFreq.end()){
            result[element] = -1;
        }else{
            result[element] = *it;
        }}
    return result; // returns the map
}
```

Pg#2

# Question#2

```
string simplifyPath(string Path){
    deque<string> directories; // using a double ended queue.
    for(size_t i=1; i<path.size(); i++){

        string currDir; //extracting name of a single directory
        while(~~curr Dir~~ i<path.size() && path[i]!='/'){
            currDir += path[i];
            i++;
        }

        if(currDir == "" || currDir == "."){ // no operation req
            continue;

        if(currDir == ".."){//go one step back
            if(!directories.empty()){ // check for non-emptiness
                ~~directories.push=back(t~~
                directories.pop_back();}}
        else{ // pushes the current directory
            directories.push_back(currDir);
        }
    }
    // calculating simplified path.
    string result="";
    for(const string &dir : directories){
        result+="/" + dir;
    }

    return result.empty()? "/" : result;

}
```

# Question # "3"



```
//using a custom structure for simplicity
struct Range {
    int min, max;
    Range(int val) : min(val), max(val) {}
};

vector<Range> maxMinFinder(vector<int> &nums, int k){
    vector<Range> result;
    for(int i=0; i<= nums.size(); i++){
        Range curr(nums[i]);
        // comparing each value in the window to find
        // minimum and maximum.
        for(int j=i+1; j < k+i; j++){
            if(nums[j] > curr.max)  // checking for max
                curr.max= nums[j];
            if(nums[j] < curr.min)  // checking for min
                curr.min = nums[j];
        }
        result.push_back(curr); // storing result in a
                                vector
    }
    return result;
}
```

Pg#4

# Question#4

```
/* Considering a ListNode struct as
   struct ListNode{
       int val;
       ListNode *next;
       ListNode *prev;
       ListNode(): val(0), next(nullptr), prev(nullptr){}
   }
*/

ListNode* reverse(ListNode* head){
// This function assumes that a valid ~~bad~~ head of
// a list is provided, and returns new head of
// the reversed list.

    ListNode* curr = head, *temp=nullptr; *prev=nullptr;

    // Traverse through the list and swap the pointers

    while (curr !=nullptr){
        //swapping mechanism
        temp=curr->next;
        curr->next=curr->prev;
        curr->prev=temp;

        // move to next node using temp as it stored next;
        prev=curr; // to catch the last node.
        curr=temp;
    }

    // since last valid node of the list is in prev
    // which is now the head.

    return prev;
}
```

# Question #5 "-"



```
/* Considering a ListNode struct as
   struct ListNode{
        int val;
        ListNode *next, *prev;
        ListNode(): val(0), next(nullptr), prev(nullptr){}
        ListNode(int x): val(x), next(nullptr), prev(nullptr){}
   };
*/

void insert(ListNode* head, int val, size_t pos){
// This function assumes that a valid head of a
// CDLL is provided.

ListNode* newNode = new ListNode(val);

// Checks if list is empty.
 if (head==nullptr){
      head = newNode;
      newNode->next = newNode;
      newNode->prev = newNode; return;
}

// Inserting at the head

if (pos==0){
      newNode->next = head;
      newNode->prev = head->prev;
      head->prev->next = newNode;
      head->prev = newNode;
      head = newNode;
      return;
}
```

Pg#6

```
// since this is a circular list, positions beyond the
// current length wraps around the list.

ListNode* curr = head;

for (size_t i = 1; i < pos ; i++){
        curr = curr->next;
}

newNode->next = curr->next;
newNode->prev = curr;
curr->prev->next
curr->next->prev = newNode;
curr->next = newNode;
}
```